NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

UCS 2312 Data Structures Lab

Assignment 4: StackADT and its application

Create an ADT for the stack data structure with the following functions. stack*ADT* will have the integer array, top and size.

[CO1, K3]

a. createStack(top) – initialize size and top with -1
b. push(top,data) – push data into the stack if stack is not full. Print a message when stack is

full

c. pop(top) – decrements the top by 1
d. peek(top)– returns the element at top, if stack is not empty, otherwise returns -1
e. isEmpty(top) – returns 1 if stack empty, otherwise returns 0
f. isFull(top) – returns 1 if stack full, otherwise returns 0

Test the operations of stackADT with the following test cases

Operation	**Expected Output**
peek(top)	Empty
push(top,2)	2
push(top,4)	4, 2
push(top,6)	6, 4, 2
push(top,8)	Full
pop(top)	
peek(top)	4
peek(top)	4
pop(top)	
pop(top)	
peek(top)	Empty
pop(top)	
pop(top)	
push(top,11)	11
peek(top)	11

Best practices to be followed:

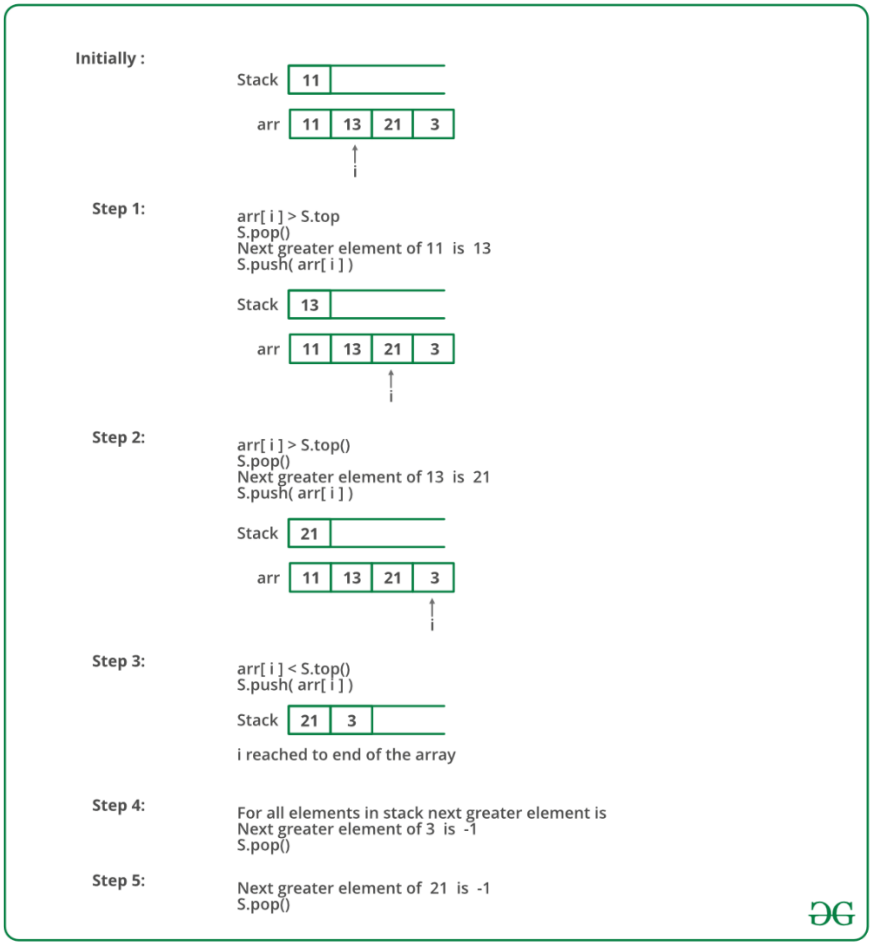
- Design before coding
- Usage of algorithm notation
- Use of multi-file C program
- Versioning of code

Application using Stack

1. Evaluate the infix to postfix expression using Stack
   Example: (2+3)*(4+5)
   Ans: 23+45+*
2. Convert the given decimal number into binary using stack
   Example: 14

Ans: 1110

3. Find next greater element using stack



Follow the steps mentioned below to implement the idea:

- Push the first element to stack.
- Pick the rest of the elements one by one and follow the following steps in the loop.
    - Mark the current element as next.
    - If the stack is not empty, compare top most element of stack with next.
    - If next is greater than the top element, Pop element from the stack. next is the next greater element for the popped element.
    - Keep popping from the stack while the popped element is smaller than next. next becomes the next greater element for all such popped elements.
- Finally, push the next in the stack.
- After the loop in step 2 is over, pop all the elements from the stack and print -1 as the next element for them.

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

ALGORITHMS:

1.FUNCTION NAME:create
INPUT:
i)struct stack *
ii)int
OUTPUT: None

1. s->size=size;
2. s->top=-1;

2.FUNCTION NAME:push
INPUT:
i)struct stack
ii)int
OUTPUT: None

1. if (isfull(s))
   DISPLAY STACK IS FULL!
2. else
   {
   s->top+=1;
   s->arr[s->top]=ele;
   DISPLAY PUSHED
   }

3.FUNCTION NAME:pop
INPUT:
i)struct stack *
OUTPUT: None

1. if (isempty(s))
   DISPLAY EMPTY!
2. else
   {
   DISPLAY s->arr[s->top]
   s->top-=1;
   }

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

4.FUNCTION NAME:peek
INPUT:
i)struct stack *
OUTPUT:int

```
1. if (isempty(s))
   return -1;
2. else
   return s->arr[s->top];
```

5.FUNCTION NAME:isfull
INPUT:
i)struct stack *
OUTPUT:int

```
1. if (s->top==s->size-1)
    return 1;
2.  else
   return 0;
```

6.FUNCTION NAME:isempty
INPUT:
i)struct stack *
OUTPUT:int

```
1. if (s->top==-1)
   return 1;
2.  else
   return 0;
```

7.FUNCTION NAME:postfix
INPUT:
i)struct stack *
ii)char []
OUTPUT: None

```
int i = 0;
char str1[100];
int k = 0;
while(str[i]!='\0')
{
   if (str[i]=='(')
   {
      push(h,str[i]);
```

```
    }
    else if (alpha(str[i]))
    {
       str1[k]=str[i];
       k+=1;
    }
    else if (str[i]==')')
    {
       int res =close(h,str1,k);
       k=res;
    }
    else
    {
       if (check(h)==-1)
          push(h,str[i]);
       else
       {
          if (operator(str[i])<=operator(check(h)))
             {
                str1[k]=pop(h);
                k++;
             }
          else
             push(h,str[i]);
       }
    }
    i++;
  }
  for (int i = 0;i<k;i++)
  {
     printf("%c",str1[i]);
  }
```

8.FUNCTION NAME:binary

INPUT:

i)int

OUTPUT: None

```
1. struct stack *s;
2. s=(struct stack *)malloc(sizeof(struct stack));
3. create(s,10000);
4. int temp=n;
5. while(n>0)
   {
```

```
        push(s,n%2);
         n=n/2;
         }
    6.  while(!isempty(s))
        {
                pop(s);
        }
```

9.FUNCTION NAME:greater

INPUT:

i)struct stack *

OUTPUT: None

```
  1.  if (s->top==-1)
 {
    s->arr[++s->top]=ele;

 }
  2.  else
      {
            if (ele<peek(s))
             {
             s->arr[++s->top]=ele;

            }
            else
            {
                  while (!isempty(s))
                  {
                        if (peek(s)<ele)
                        {
                         printf("\n%d has greater element %d",peek(s),ele);
                        pop(s);
                        }
                        else
                        {break;}
                   }
                  s->arr[++s->top]=ele;
            }

      }
```

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

CODE:

Stackadt.h

```
#include <stdio.h>
#include <stdlib.h>

struct stack
{   int size;
   int top;
   int arr[100];
};

void create(struct stack *s,int size)
{
   s->size=size;
   s->top=-1;
}

int isfull(struct stack *s)
{
   if (s->top==s->size-1)
      return 1;
   else
      return 0;
}

void push(struct stack *s,int ele)
{
   if (isfull(s))
      printf("STACK IS FULL!\n");
   else
   {
      s->top+=1;
      s->arr[s->top]=ele;

   }
}

int isempty(struct stack *s)
{
   if (s->top==-1)
      return 1;
   else
      return 0;
}
```

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

```
void pop(struct stack *s)
{
   if (isempty(s))
      printf("EMPTY!\n");
   else
   {printf("%d ",s->arr[s->top]);
      s->top-=1;}

}

int peek(struct stack *s)
{
   if (isempty(s))
      return -1;
   else
      return s->arr[s->top];
}

void binary(int n)
{
   struct stack *s;
   s=(struct stack *)malloc(sizeof(struct stack));
   create(s,10000);
   int temp=n;
   while(n>0)
   {
      push(s,n%2);
      n=n/2;
   }
   while(!isempty(s))
   {
      pop(s);
   }
}


void push(struct stack *s,int ele)
{

   if (s->top==-1)
   {
      s->arr[++s->top]=ele;

   }
   else
```

```
   {
      if (ele<peek(s))
      {
         s->arr[++s->top]=ele;

      }
      else
      {
         while (!isempty(s))
         {
            if (peek(s)<ele)
             {
                printf("\n%d has greater element %d",peek(s),ele);
               pop(s);
               }
            else
               {break;}
         }
         s->arr[++s->top]=ele;
      }

   }

}
int close(struct stack *h,char str1[],int k)
{
   int i = h->top;
   int j = 1;
   while(h->arr[i]!='(')
   {
      str1[k]=h->arr[i];
      i--;
      k++;
      j++;
   }
   h->top=h->top-j;
   return k;
}

char check(struct stack *h)
{
   if (h->top=='(')
      return -1;
   else
   return h->arr[h->top];
}
```

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

```
int operator(char data)
{
   if (data=='+'|| data == '-')
      return 1;
   else if (data == '*' || data=='/')
      return 2;
   else
      return 0;
}

void conv(struct stack *h,char str[])
{
   int i = 0;
   char str1[100];
   int k = 0;
   while(str[i]!='\0')
   {
      if (str[i]=='(')
      {
         push(h,str[i]);
      }
      else if (alpha(str[i]))
      {
         str1[k]=str[i];
         printf("%c\n",str1[k]);
         k+=1;
      }
      else if (str[i]==')')
      {
         int res =close(h,str1,k);
         k=res;
      }
      else
      {
         if (check(h)==-1)
            push(h,str[i]);
         else
         {
            if (operator(str[i])<=operator(check(h)))
               {
                  str1[k]=pop(h);
                  k++;
               }
            else
               push(h,str[i]);
         }
      }
```

```
        i++;
    }
    for (int i = 0;i<k;i++)
    {
        printf("%c",str1[i]);
    }
}
```

Main.c

```
#include <stdio.h>
#include <stdlib.h>
#include "stackadt.h"

void main()
{
    struct stack *s;
    s=(struct stack *)malloc(sizeof(struct stack));
    create(s,4);
    push(s,1);
    push(s,2);
    push(s,3);
    push(s,4);
    push(s,5);
    pop(s);
    pop(s);
    pop(s);
    printf("PEEK : %d",peek(s));
    printf("\n---BINARY----\n");
    printf("enter the decimal number");
    int n;
    scanf("%d",&n);
    binary(n);

     struct stack *h;
    h=(struct stack *)malloc(sizeof(struct stack));
    printf("enter the string ");
    char str[100];
    scanf("%[^\n]s",str);
    init(h);
    conv(h,str);

    struct stack *s;
    s=(struct stack *)malloc(sizeof(struct stack));
    init(s);
    printf("elements :");
    int n;
```

```
    scanf("%d",&n);
    for (int i =0;i<n;i++)
    {
        printf("\nenter element :");
        int a;
        scanf("%d",&a);
        push(s,a);
    }

    while(!isempty(s))
    {
        printf("\n%d has no greater element ",pop(s));
    }
}
```

OUTPUT:

```
C:\Users\User\OneDrive\SEM3\DATA STRUCTURES\PRACTICE\STACK>f1.exe
STACK IS FULL!
4 3 2 PEEK : 1
---BINARY----
enter the decimal number8
1 0 0 0
```

```
enter the string ((((a+b)*c)/d)+f)
a
b
c
d
f
ab+c*d/f+
```

```
elements :5

enter element :50

enter element :5

enter element :4

enter element :3

enter element :15

3 has greater element 15
4 has greater element 15
5 has greater element 15
15 has no greater element
50 has no greater element
```

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE - B

QUESTION 2: Evaluate the postfix expression using stacks

CODE:

```
int isdigi(char data)
{
   int asc=(int)data;
   if ((asc >=48) && (asc <= 57) )
   {
      return 1;
   }
   else
   {
      return 0;
   }
}

int whichop(char data)
{
   if (data=='+')
      return 1;
   else if (data=='-')
      return 2;
   else if (data=='*')
      return 3;
   else if (data == '/')
      return 4;
}

int evalpost(struct stack *s,char arr[])
{
   int i =0;
   while(arr[i]!='\0')
   {
      if (isdigi(arr[i]))
      {
         push(s,((int)arr[i]-48));
      }
      else
      {
         int ch = whichop(arr[i]);
         if (ch == 1)
         {
            int a = pop(s);
            int b = pop(s);

            push(s,b+a);
```

```
        }
        else if (ch==2)
        {
           int a = pop(s);
           int b = pop(s);
           push(s,b-a);
        }
        else if (ch == 3)
        {
           int a = pop(s);
           int b = pop(s);
           push(s,b*a);
        }
        else
        {
           int a = pop(s);
           int b = pop(s);
           push(s,b/a);
        }
     }
     i++;
  }
  return (pop(s));
}
```

OUTPUT:

```
C:\Users\User\OneDrive\SEM3\DATA STRUCTURES\PRACTICE\STACK>f1.exe
enter the postfix expression :23+4-5*
ANSWER : 5
```

QUESTION 3: Find minimum of the stack in O(1) time complexity. You can use another stack.

CODE:

```
int min(struct stack *mh)
{
  return mh->arr[mh->top];
}

int isempty(struct stack *h)
{
  if (h->top==-1)
     return 1;
  else
     return 0;
```

```
}

int pop(struct stack *h,struct stack *mh)
{
   if (!isempty(h))
   {
      h->top-=1;
      mh->top-=1;
      printf("\n MINIMUM : %d\n",min(mh));
   }
   else
   {
      return -1;
   }
}
```

OUTPUT:

```
enter the number of elements : 5

---PUSH---
enter the element : 1

 MINIMUM : 1
enter the element : 2

 MINIMUM : 1
enter the element : 3

 MINIMUM : 1
enter the element : -1

 MINIMUM : -1
enter the element : 5

 MINIMUM : -1

---POP---

 MINIMUM : -1

 MINIMUM : 1

 MINIMUM : 1

 MINIMUM : 1
```

NAME : S.KARTHIKEYAN **3122 22 5001 056**
CSE - B